You said:

codigo(1).c
Code

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Worked for 18s

ChatGPT said:

Análise de segurança do arquivo `codigo(1).c`

MALWARE IDENTIFICADO

Classificação principal: Keylogger (Spyware) para Linux, executado em nível de kernel.

O código implementa um módulo do kernel Linux cuja finalidade é capturar as teclas pressionadas pelo usuário, armazená-las em memória e disponibilizá-las para leitura por meio de um dispositivo de caractere.

1. Identificação do programa

O arquivo está escrito em C e utiliza bibliotecas específicas do kernel Linux:

```
#include <linux/module.h>
#include <linux/keyboard.h>
#include <linux/input.h>
#include <linux/fs.h>
```

Essas bibliotecas permitem desenvolver módulos que interagem diretamente com o kernel, incluindo o subsistema de teclado e os dispositivos de entrada.

O código também contém:

```
MODULE_LICENSE("GPL");
MODULE_AUTHOR("Ciaran McNally - maK");
#define DEVICE_NAME "maK_it"
#define DEVICE_MAJOR 33
```

Isso identifica o programa como um módulo de kernel chamado `maK_it`. A declaração de licença GPL não significa que o programa seja benigno.

codigo.c

2. Captura das teclas pressionadas

O trecho mais importante está na função `key_notify()`:

```
int key_notify(struct notifier_block *nblock,
               unsigned long kcode, void *p) {
    struct keyboard_notifier_param *param = p;

    if (kcode == KBD_KEYCODE) {
        ...
    }
}
```

Essa função recebe notificações do subsistema de teclado do Linux.

O código verifica os eventos de teclado e identifica quando uma tecla é pressionada:

```
if(param->down) {
    ...
}
```

Isso significa que o programa captura os eventos de pressionamento das teclas, em vez de simplesmente fornecer uma funcionalidade convencional de entrada de teclado.

codigo.c

3. Identificação das teclas

O programa possui duas tabelas:

```
static const char* keys[] = {...};
static const char* keysShift[] = {...};
```

A primeira representa os caracteres em condições normais. A segunda representa os caracteres quando a tecla Shift está pressionada.

Exemplos de caracteres reconhecidos incluem letras, números, símbolos e teclas especiais, como Enter, Esc, Backspace e Tab.

O código também monitora o estado da tecla Shift:

```
if(param->value == 42 || param->value == 54) {
    ...
    shiftPressed = 1;
}
```

Dessa forma, tenta registrar os caracteres de acordo com a tecla Shift.

codigo.c

4. Armazenamento das informações capturadas

O código declara um buffer global:

```
char keyBuffer[1000000];
```

Esse espaço de memória permite armazenar até aproximadamente um milhão de caracteres.

Quando uma tecla é identificada, seu caractere correspondente é gravado no buffer:

```
*basePtr = c;
basePtr++;
```

Quando o ponteiro chega ao limite definido, ele retorna ao início do buffer:

```
if(basePtr == endPtr) {
    basePtr = keyBuffer;
}
```

Portanto, o programa mantém os registros das teclas pressionadas em memória, sobrescrevendo dados anteriores quando o espaço disponível é esgotado.

codigo.c codigo.c

5. Disponibilização dos registros capturados

Outro comportamento importante está na função `read_dev()`:

```
result = copy_to_user(buf, keyBuffer, key);
```

Essa instrução copia os registros armazenados no buffer para um espaço de memória pertencente a um programa em espaço de usuário.

O programa também registra um dispositivo de caractere:

```
major = register_chrdev(DEVICE_MAJOR, DEVICE_NAME, &fops);
```

O dispositivo utiliza o nome `maK_it` e o número principal 33.

Isso fornece uma interface pela qual outro programa pode consultar os dados capturados. A criação efetiva de um nó de dispositivo em `/dev` pode exigir uma configuração adicional.

codigo.c codigo.c

6. Ativação e desativação do módulo

Na inicialização, o programa executa:

```
register_keyboard_notifier(&nb);
```

Essa chamada registra sua função de monitoramento no subsistema de teclado.

Na desativação, executa:

```
unregister_keyboard_notifier(&nb);
unregister_chrdev(DEVICE_MAJOR, DEVICE_NAME);
memset(keyBuffer, 0, sizeof(keyBuffer));
```

Assim, deixa de monitorar o teclado, remove o registro do dispositivo e limpa o buffer de memória.

codigo.c

7. Indicadores de comportamento malicioso

Os comportamentos identificados são:

Indicador	Resultado
Monitoramento de eventos do teclado	Detectado
Captura de teclas pressionadas	Detectado
Armazenamento das teclas em memória	Detectado
Disponibilização dos registros para leitura	Detectado
Execução como módulo do kernel	Detectado
Envio dos registros pela rede	Não identificado
Comunicação com servidor C2	Não identificada
Persistência após reinicialização	Não identificada
Ocultação de processos ou arquivos	Não identificada

Uma observação importante: não há evidência, neste código, de transmissão dos registros pela rede. A captura e o armazenamento das teclas, entretanto, estão efetivamente implementados.

Também existem problemas de implementação, como o registro do notificador antes da inicialização do semáforo, o que pode provocar falhas em determinadas condições de execução.

8. Veredito final

MALWARE

Categoria: Keylogger.

Subcategoria: Spyware / Capturador de teclas em nível de kernel Linux.

Confiança da classificação: Alta.

Justificativa: O programa monitora eventos do teclado, captura os caracteres digitados, armazena os registros em memória e disponibiliza esses dados para leitura por meio de um dispositivo de caractere.

Conclusão: O arquivo `codigo(1).c` não deve ser considerado um programa benigno. Sua funcionalidade central é a implementação de um keylogger para Linux. Embora não apresente mecanismos próprios de comunicação com servidores externos ou persistência, sua capacidade de capturar informações digitadas representa comportamento característico de spyware.

Recomendação: Não compile nem carregue esse módulo em um sistema operacional de produção.

You said:

Qual é a categoria principal deste malware?

ChatGPT said:

A categoria principal deste malware é **Keylogger**, pertencente à família de **Spyware**.

Ele captura as teclas digitadas pelo usuário em sistemas Linux, armazenando os registros em memória para posterior leitura.